In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv(r"C:\Users\Public\Downloads\Mall_Customers.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 7.9 KB
```

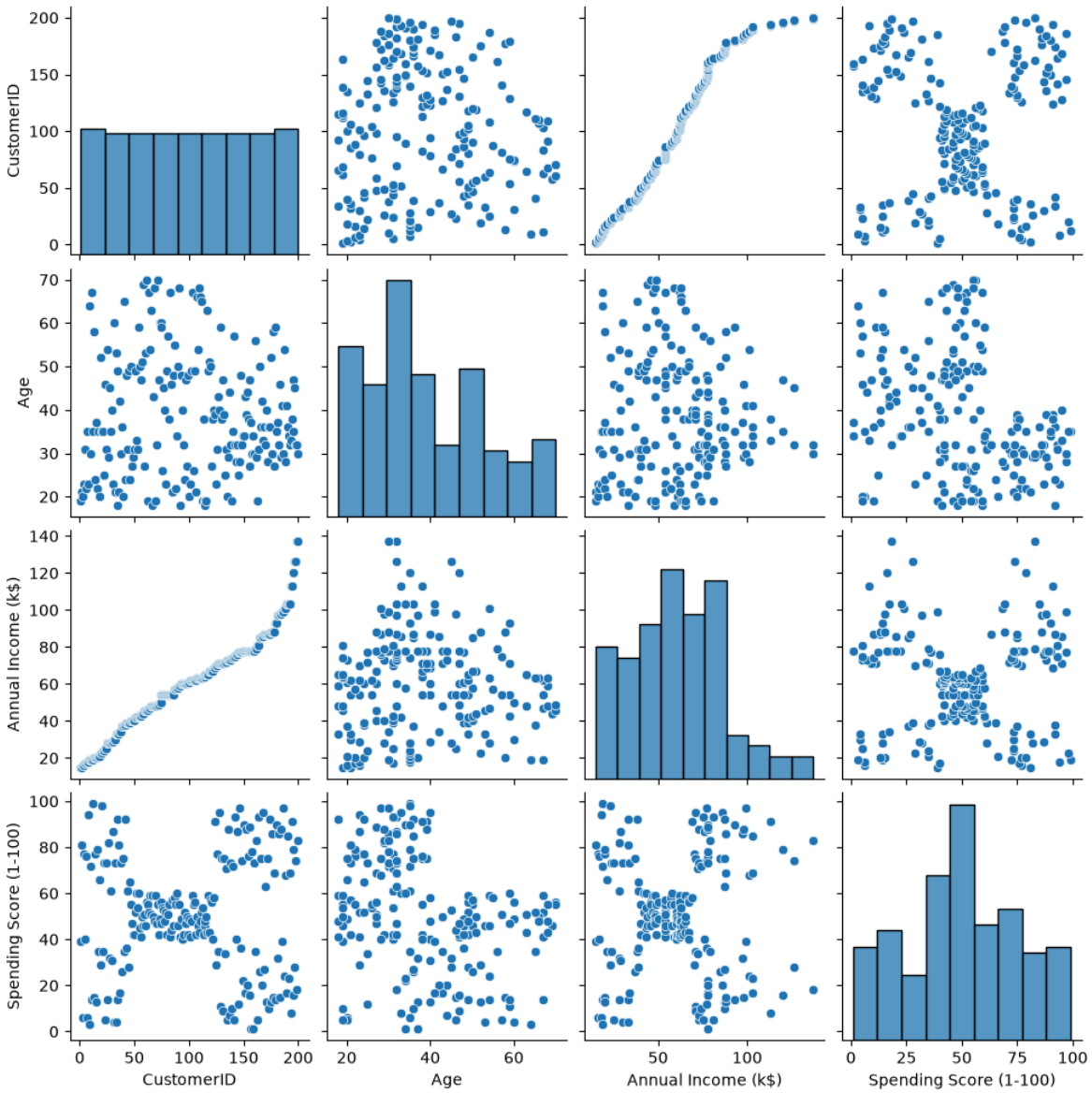
In [2]:
```
df.head()
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Gender</th> <th>Age</th> <th>Annual Income (k$)</th> <th>Spending Score (1-100)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1		Male	19	15	39
2		Male	21	15	81
3	Female	20	16	6	
4	Female	23	16	77	
5	Female	31	17	40	

In [3]:
```
sns.pairplot(df)
```

Out[3]: <seaborn.axisgrid.PairGrid at 0x27c05a4b0e0>



In [3]:
```
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

Out[3]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-

link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table

.parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-

color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>
KMeans(n_clusters=5)
**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label sk-toggleable__label-arrow">
KMeans
?Documentation for KMeansiNot fitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy n_clusters to clipboard" onclick="copyToClipboard('n_clusters', this.parentElement.nextElementSibling)" ></button>	n_clusters n_clusters: int, default=8 Th number of centroids to generate. Fo to :ref:`sphx_glr_auto_examples_cl
<button type="button" class="copy-paste-icon" aria-label="Copy init to clipboard" onclick="copyToClipboard('init', this.parentElement.nextElementSibling)" ></button>	init init: {'k-means++', 'random'}, callable or default='k-means++' Method for initializa cluster centroids using sampling based on an emp contribution to the overall inertia. This technic implemented is "greedy k-means++". It differs fro trials at each sampling step and choosing 'random': choose `n_clusters` observation centroids. * If an array is passed, it shou gives the initial centers. * If a call n_clusters and a random state and return an initia see :ref:`sphx_glr_auto_examples_clus evalu example :ref:`sphx_glr_auto_examples_cluster
<button type="button" class="copy-paste-icon" aria-label="Copy n_init to clipboard" onclick="copyToClipboard('n_init',	n_init n_init: 'auto' or int, default='auto' run with different centroid seeds. T consecutive runs in terms of inertia. Several ru dimensional problems (see :ref:`kmeans_spars the number of runs depends on the value of ini

this.parentElement.nextElementSibling)" ></button>	callable; 1 if using `init='k-means++'` or `ini 1.2  Added 'auto' option for `n_init`. ..
<button type="button" class="copy-paste-icon" aria-label="Copy max_iter to clipboard" onclick="copyToClipboard('max_iter', this.parentElement.nextElementSibling)" ></button>	max_iter max_iter: int, default=300 Max
<button type="button" class="copy-paste-icon" aria-label="Copy tol to clipboard" onclick="copyToClipboard('tol', this.parentElement.nextElementSibling)" ></button>	tol tol: float, default=1e-4 Relative tolera difference in the clus
<button type="button" class="copy-paste-icon" aria-label="Copy verbose to clipboard" onclick="copyToClipboard('verbose', this.parentElement.nextElementSibling)" ></button>	verbose verbo
<button type="button" class="copy-paste-icon" aria-label="Copy random_state to clipboard" onclick="copyToClipboard('random_state', this.parentElement.nextElementSibling)" ></button>	random_state random_state: int, Randon Determines random number generation make the randomness deterministic
<button type="button" class="copy-paste-icon" aria-label="Copy copy_x to clipboard" onclick="copyToClipboard('copy_x', this.parentElement.nextElementSibling)" ></button>	copy_x copy_x: bool, default=True <br numerically accurate to center the data first. data is not modified. If False, the original da function returns, but small numerical differences then adding the data mean. Note that if the o be made even if copy_x is False. If the ori
<button type="button" class="copy-paste-icon" aria-label="Copy algorithm to clipboard" onclick="copyToClipboard('algorithm', this.parentElement.nextElementSibling)" ></button>	algorithm algorithm: {"lloyd", "elkan"}, default= The classical EM-style algorithm is `"lloyd"`. T on some datasets with well-defined cluster it's more memory intensi shape `(n_samples, n_clusters)`. <br algorithm .. versionchanged:: 1.1<br "auto" and "full". Chan

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', '');

```
navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "${row}"`); const formattedText = `[\n${rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ”', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ”'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based
```

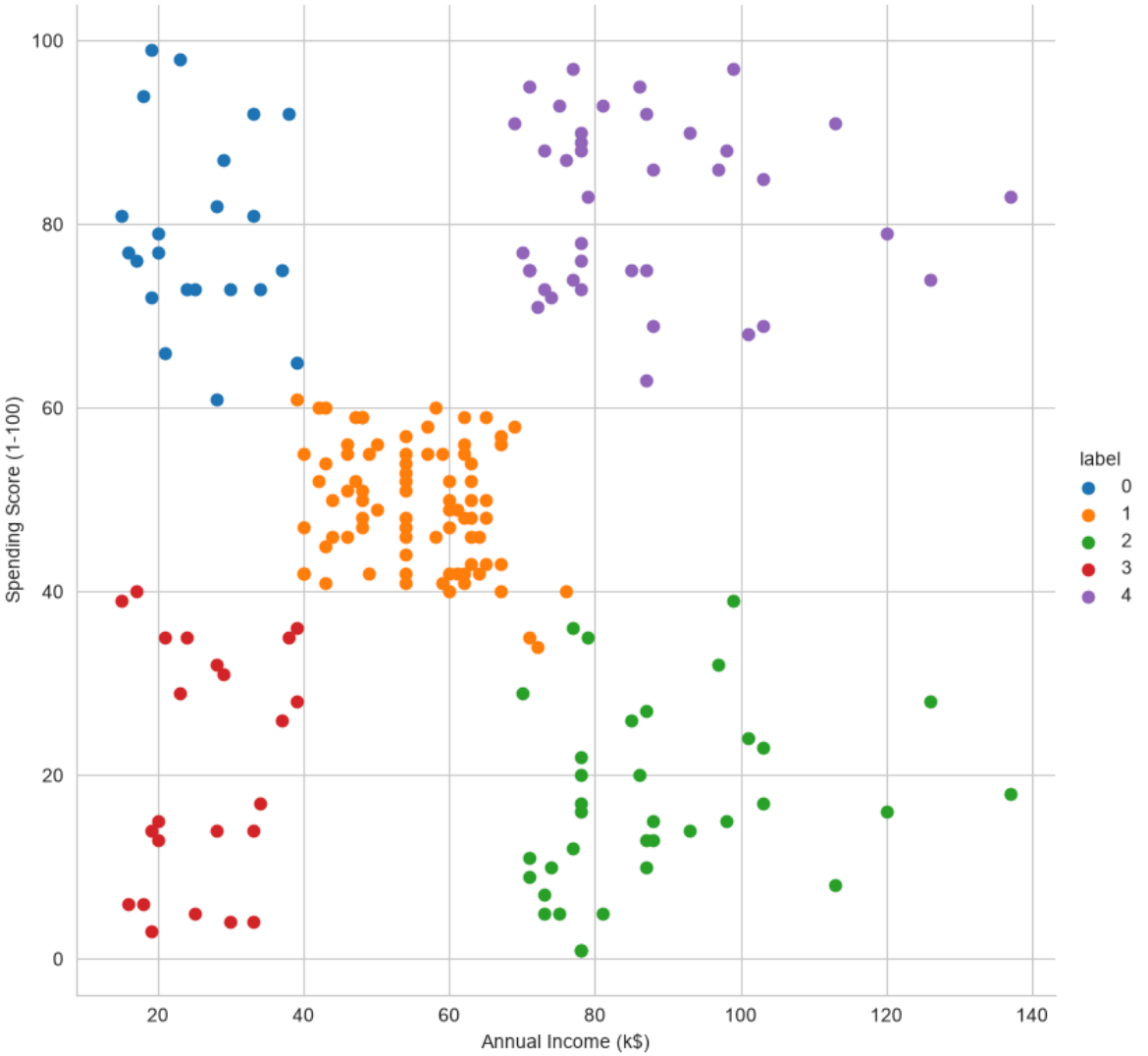
on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s*\(\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\)\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [5]:
```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Annual Income (k$)</th> <th>Spending Score (1-100)</th> <th>label</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

15	39	3
15	81	0
16	6	3
16	77	0
17	40	3

In [6]:
```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```



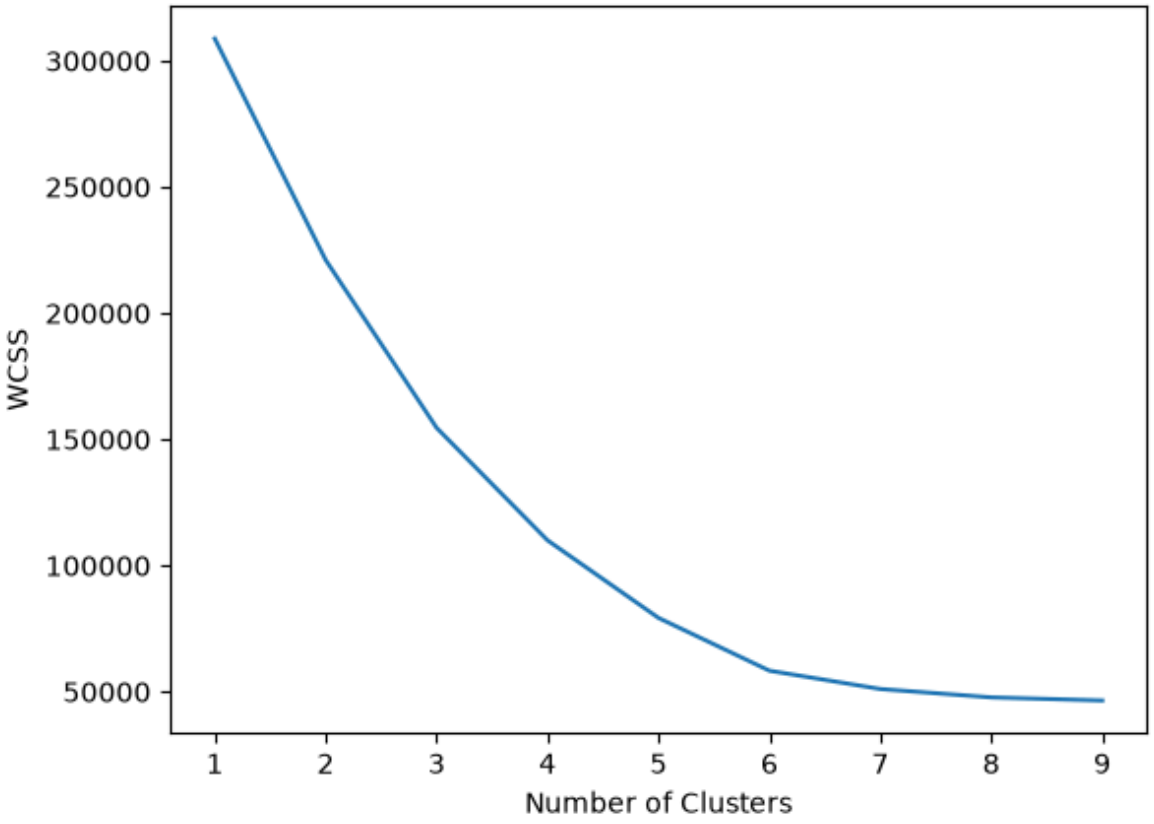
In [5]:
```
features_el = df.iloc[:, [2, 3, 4]].values

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []

for i in range(1, 10):
    model = KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1, 10), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```


300000
250000
200000
150000
100000
50000
WCSS
1
2
3
4
5
6
7
8
9
Number of Clusters

In [ ]: